

Snake Game

Project Documentation

How it was built and how it works

Technology	HTML5, CSS3, vanilla JavaScript (no libraries or frameworks)
Files	index.html, style.css, script.js
Storage	Browser localStorage (high score)
Controls	Arrow keys

1. Overview

This project is a browser-based version of the classic Snake game, written with plain HTML, CSS and JavaScript. The player steers a snake around a grid with the arrow keys, eats red food blocks to score points and grow longer, and loses when the snake runs into a wall. No installation, build step or external library is needed - the game runs by opening **index.html** in a browser.

Features

- Full-screen grid board that is generated automatically to fit the browser window
- Start screen and Game Over screen shown as blurred overlay modals
- Live score (10 points per food), running timer and a persistent high score
- High score saved in the browser so it survives page reloads
- Restart button to play again without reloading the page

Technology stack

Layer	Technology	Role in the project
Structure	HTML5	Score panel, empty board container and the start / game-over modals
Styling	CSS3 (Grid, Flexbox, custom properties)	Dark theme, grid board layout, modal overlay with backdrop blur
Logic	Vanilla JavaScript (ES6)	Board generation, game loop, movement, collisions, scoring, timer
Persistence	Web Storage API (localStorage)	Saves the high score between sessions

2. Project Structure

```
snake-game/  
|-- index.html  page markup (score panel, board, modals)  
|-- style.css   theme, layout, board cells, modal styling  
|-- script.js   all game logic
```

File	Responsibility
index.html	Defines the three info boxes (High Score, Score, Time), an empty <code>.board</code> div that JavaScript fills with cells, and a <code>.modal</code> containing the start and game-over screens.
style.css	Defines CSS variables (colours, spacing, radii), the page layout, the grid on the board, the look of each cell, and the full-screen modal.
script.js	Builds the board, runs the game loop, handles keyboard input, detects collisions, updates score/time and manages the modals.

3. How the Project Was Built

The project was built in layers - markup first, then styling, then behaviour - which is a natural order for a small browser game. The steps below describe that process as it can be read from the code.

Step 1 - Page skeleton (index.html)

A single `<main>` element holds two parts: a `<section>` with the score panel and the board, and a `.modal` overlay. The board is deliberately empty in the HTML; its cells are created by JavaScript so the grid size can adapt to the screen. The spans with ids `high-score`, `score` and `time` are the hooks that the script updates.

Step 2 - Styling (style.css)

- A reset (`margin: 0; padding: 0; box-sizing: border-box`) and full-height `html`, `body`.
- CSS custom properties in `:root` hold the colour palette, spacing scale, border colour and border radii, so the theme can be changed in one place.
- The `section` is a flex column: info boxes on top, board filling the remaining height with `flex-grow: 1`.
- The board uses CSS Grid with `repeat(auto-fill, minmax(50px, 1fr))` for both columns and rows, giving cells of roughly 50 px.
- Each `.block` is a bordered circle; the `.fill` class (white) marks the snake and `.food` (red) marks the food.
- The `.modal` is fixed, covers the viewport, uses a translucent background with `backdrop-filter: blur(4px)`, and centres its content with Flexbox. The game-over panel is hidden by default.

Step 3 - Game logic (script.js)

The script was then written in this order: (1) select all needed DOM elements, (2) calculate the grid size and create the cells, (3) define the game state (snake, food, direction, score, time), (4) write the `render()` function that advances the game by one step, (5) connect the Start and Restart buttons, and (6) add the keyboard listener for steering.

4. How It Works

4.1 Board generation

The number of columns and rows is calculated from the real pixel size of the board divided by the block size (50 px). Two nested loops then create one `div.block` for each cell and store a reference to it in the `blocks` object, keyed by "row-col". This lookup table lets the game paint any cell instantly with `blocks["3-5"]` instead of searching the DOM.

```
const cols = Math.floor(board.clientWidth / blockWidth);
const rows = Math.floor(board.clientHeight / blockHeight);

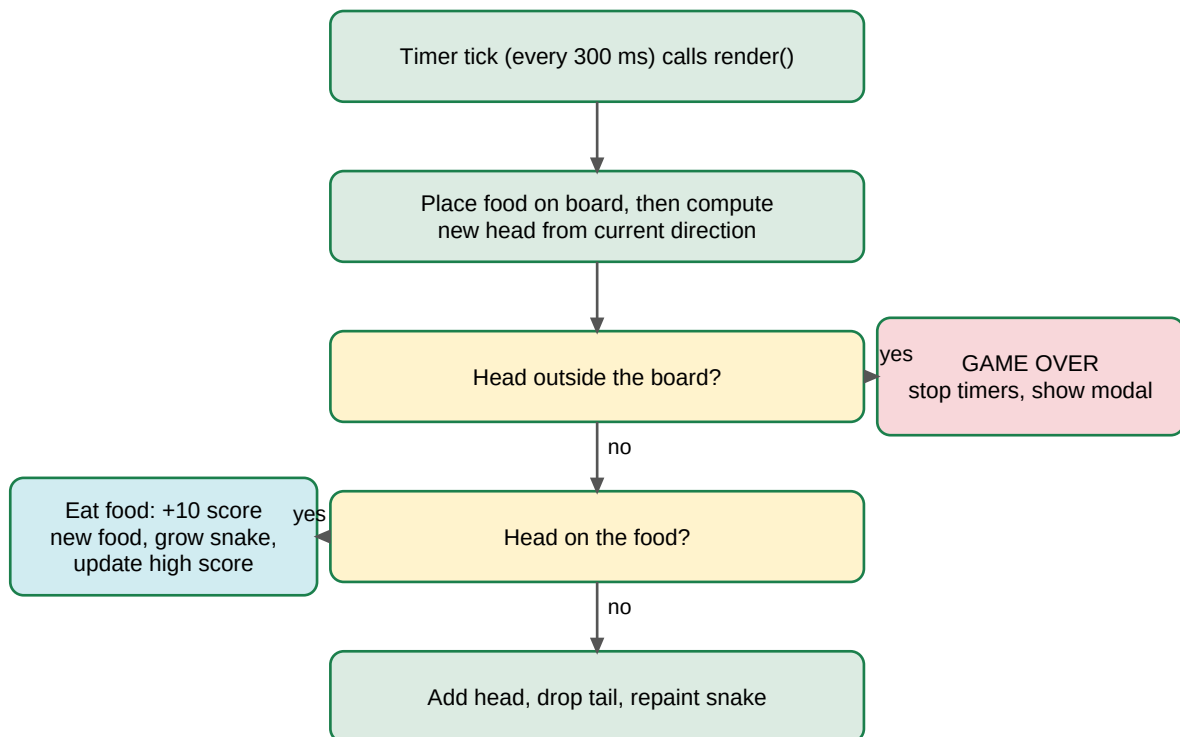
for (let row = 0; row < rows; row++) {
  for (let col = 0; col < cols; col++) {
    const block = document.createElement("div");
    block.classList.add("block");
    board.appendChild(block);
    blocks[`${row}-${col}`] = block;
  }
}
```

4.2 Game state

Variable	Meaning
snake	Array of {x, y} segments. Index 0 is the head. x is the row and y is the column. Starts as one segment at (1, 3).
food	A single {x, y} position chosen randomly inside the grid.
direction	One of "up", "down", "left", "right". Starts as "right".
score / highScore	Current score and best score. highScore is read from localStorage at start-up.
time	Elapsed time stored as text in the form "min-sec".
intervalId / timerIntervalId	Handles of the two setInterval timers (game loop and clock) so they can be stopped.

4.3 The game loop

When the player clicks **Start Game**, a `setInterval` calls `render()` every 300 milliseconds. Each call is one 'tick' of the game and follows the flow below.



4.4 Movement

Every tick, a new head position is calculated from the current head and the direction. Moving left or right changes the column (y), moving up or down changes the row (x).

```

if (direction === "left")    head = { x: snake[0].x,    y: snake[0].y - 1 };
else if (direction === "right") head = { x: snake[0].x,    y: snake[0].y + 1 };
else if (direction === "down") head = { x: snake[0].x + 1, y: snake[0].y };
else if (direction === "up")   head = { x: snake[0].x - 1, y: snake[0].y };

```

The snake is then moved with a simple trick: the old snake cells are un-painted, the new head is added to the front of the array with `unshift`, the last segment is removed with `pop`, and all cells are painted again. The body therefore follows the head without any extra bookkeeping.

```

snake.forEach(s => blocks[`${s.x}-${s.y}`].classList.remove("fill"));
snake.unshift(head);
snake.pop();
snake.forEach(s => blocks[`${s.x}-${s.y}`].classList.add("fill"));

```

4.5 Wall collision and Game Over

If the new head is outside the grid (negative, or equal to or larger than the number of rows/columns), both timers are cleared, the modal is shown, the start panel is hidden and the game-over panel is displayed. The function returns early, so the snake does not move into the wall.

```

if (head.x < 0 || head.x >= rows || head.y < 0 || head.y >= cols) {
  clearInterval(intervalId);
  clearInterval(timerIntervalId);
  modal.style.display = "flex";
  startGameModal.style.display = "none";
  gameOverModal.style.display = "flex";
  return;
}

```

4.6 Eating food and scoring

When the new head lands on the food cell, the old food is removed, a new random food position is chosen and painted, and the head is added to the snake an extra time. Because the normal move step then removes only one tail segment, the snake ends up one segment longer. The score increases by 10 and, if it beats the stored record, the high score is updated and saved with `localStorage.setItem`.

```

if (head.x == food.x && head.y == food.y) {
  blocks[`${food.x}-${food.y}`].classList.remove("food");
  food = { x: Math.floor(Math.random() * rows),
    y: Math.floor(Math.random() * cols) };
  blocks[`${food.x}-${food.y}`].classList.add("food");
  snake.unshift(head);

  score += 10;
  scoreElement.innerHTML = score;
  if (score > highScore) {
    highScore = score;
    localStorage.setItem("highScore", highScore.toString());
  }
}

```

4.7 Timer

A second interval runs once per second. It splits the stored time string, increments the seconds, rolls over to the next minute at 59 and writes the result to the Time box.

4.8 Keyboard control

A global keydown listener changes the `direction` variable when an arrow key is pressed. The snake does not move immediately; the new direction is picked up on the next tick of the game loop.

Key	Effect
Arrow Up	direction = "up"
Arrow Down	direction = "down"
Arrow Left	direction = "left"
Arrow Right	direction = "right"

4.9 Start and Restart

Start Game hides the modal and starts both timers. **Restart Game** removes the old food and snake from the board, resets score, time and position, sets a new random food, hides the modal and restarts the game loop. The direction after a restart is set to "down".

5. Setup and Usage

How to run

- Place **index.html**, **style.css** and **script.js** in the same folder.
- Open **index.html** in any modern browser (Chrome, Edge, Firefox, Safari). No server is required.
- Click **Start Game**, then steer with the arrow keys.
- Eat the red blocks to score. Hitting a wall ends the game; click **Restart Game** to play again.

Customisation

What to change	Where
Game speed	The 300 in the two <code>setInterval(. . . , 300)</code> calls in <code>script.js</code> (lower is faster).
Cell size	<code>blockWidth / blockHeight</code> in <code>script.js</code> and the <code>minmax(50px, 1fr)</code> values in the <code>.board</code> rule of <code>style.css</code> . Keep them equal.
Points per food	<code>score += 10</code> in <code>script.js</code> .
Colours	The CSS variables at the top of <code>style.css</code> (snake, food, background, text, border).
Starting position	The snake array in <code>script.js</code> (and in <code>restartGame</code>).

6. Known Limitations and Possible Improvements

Reading the code closely shows several areas that behave differently from a full Snake game. They are listed here with a suggested fix, so they can be handled as next steps.

Observation	Suggested improvement
The snake can run through its own body; only walls end the game.	Check whether the new head matches any existing segment and trigger Game Over.
The snake can reverse direction instantly (e.g. right then left), which would fold it onto itself.	Ignore a key press that is the exact opposite of the current direction.
After Restart, the clock does not start again, because <code>restartGame()</code> starts the game loop but not the timer interval.	Start <code>timerIntervalId</code> inside <code>restartGame()</code> as in the Start handler.
The timer text is built as "min-sec" without leading zeros (e.g. 0-5) while the page shows 00:00 as its format.	Pad minutes and seconds to two digits and use a colon separator.
Food is placed randomly and may appear on top of the snake.	Regenerate the food position until it is on a free cell.
Several quick key presses between ticks are collapsed into the last one, so a fast U-turn is possible.	Queue direction changes and apply one per tick.
The grid size is computed once at load, so resizing the window leaves the board out of sync.	Recalculate the grid on resize, or fix the board size.

Observation	Suggested improvement
CSS: transition: all 0.3 ease is missing the unit 's', so the hover animation is ignored.	Change it to 0.3s ease.
No keyboard or touch alternative for mobile devices.	Add on-screen buttons or swipe detection; add pause/resume.

Summary. The project is a compact, dependency-free game that shows core front-end skills: DOM generation, CSS Grid and Flexbox layout, timed loops with setInterval, keyboard events, state management and localStorage persistence. With the small fixes above it would behave like a complete Snake game.